mechanics-ai999x: ONNX formula pack guide

Version 1.0.1. 102 exact formula graphs — every certified closed-form capability of mechanics-ai999x, compiled from its exact formula (nothing learned, nothing approximate). Float64, standard ONNX operators; run them with onnxruntime in Python, C++, C#, Java or the browser. Creator / engineer: Ibnul Jaif Farabi · AI999X · Apache-2.0.

Find the right formula (autocomplete)

Never guess a formula: search the catalog. `catalog.json` lists every graph with its name, keywords, inputs (meaning and SI unit), output unit and LaTeX. In JavaScript, `formula-pack.js` does the search and runs the graph; `formula-pack.d.ts` gives your editor every id and input name.

```
import { loadPack } from "./formula-pack.js";
const pack = await loadPack("https://huggingface.co/AI999X/mechanics-
ai999x/resolve/main/onnx/formulas");
pack.search("amplitude-reso");      // autocomplete: best matches first
const r = await pack.solve("amplitude_resonance_frequency.b", { wr: 2.0, k: 2.0, m: 2.0 });
console.log(r.value, r.unit);
```

Python and C#

```
import json, numpy as np, onnxruntime as ort
cat = json.load(open('onnx/formulas/catalog.json'))
e = next(x for x in cat['entries'] if x['id'] == 'amplitude_resonance_frequency.b')
s = ort.InferenceSession('onnx/formulas/' + e['file'])
print(s.run(None, {n: np.array(2.0) for n in e['inputs']}))
```

```
using Microsoft.ML.OnnxRuntime; using Microsoft.ML.OnnxRuntime.Tensors;
using var s = new InferenceSession("onnx/formulas/amplitude_resonance_frequency__b.onnx");
var inputs = new[] { NamedOnnxValue.CreateFromTensor("wr", new DenseTensor<double>(new[] { 2.0 },
new int[0])), NamedOnnxValue.CreateFromTensor("k", new DenseTensor<double>(new[] { 2.0 }, new
int[0])), NamedOnnxValue.CreateFromTensor("m", new DenseTensor<double>(new[] { 2.0 }, new int[0]))
};
using var r = s.Run(inputs);
```

Rules

- Inputs are numbers in **SI units** (convert first: 72 km/h = 20 m/s).
- A formula with several solutions returns one output per branch; take the branch whose sign the quantity allows (the entry's `sign`; formula-pack.js does it). Two valid branches means the question is ambiguous.
- A formula applies only under its stated conditions (`conditions`): a pendulum formula is for a pendulum.
- Questions in words: use the Python package (`pip install mechanics-ai999x`), which reads them and refuses what it cannot verify.

Catalogue

Amplitude-resonance frequency of a driven damped oscillator — solve for viscous damping coefficient — `amplitude_resonance_frequency.b`

$$\omega_r = \sqrt{\frac{k}{m} - \frac{b^2}{2m^2}}$$

input	meaning	unit
`wr`	driving frequency of maximum amplitude	`rad/s`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`

- **Output unit:** `kg/s`
- **File:** `amplitude_resonance_frequency__b.onnx`
- **Applies when:** driven

```
inputs {'wr': 0.5, 'k': 0.6, 'm': 0.7}
-> 0.771362 kg/s   (ran on amplitude_resonance_frequency__b.onnx; matches the Python engine)
```

Amplitude-resonance frequency of a driven damped oscillator — solve for spring constant — `amplitude_resonance_frequency.k`

$$\omega_r = \sqrt{\frac{k}{m} - \frac{b^2}{2m^2}}$$

input	meaning	unit
`wr`	driving frequency of maximum amplitude	`rad/s`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `N/m`
- **File:** `amplitude_resonance_frequency__k.onnx`
- **Applies when:** driven

```
inputs {'wr': 2.0, 'm': 2.1, 'b': 2.2}
-> 9.55238 N/m   (ran on amplitude_resonance_frequency__k.onnx; matches the Python engine)
```

Amplitude-resonance frequency of a driven damped oscillator — solve for oscillating mass — `amplitude_resonance_frequency.m`

$$\omega_r = \sqrt{\frac{k}{m} - \frac{b^2}{2m^2}}$$

input	meaning	unit
`wr`	driving frequency of maximum amplitude	`rad/s`
`k`	spring constant	`N/m`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `kg`
- **File:** `amplitude_resonance_frequency__m.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** driven

Amplitude-resonance frequency of a driven damped oscillator — solve for driving frequency of maximum amplitude — `amplitude_resonance_frequency.wr`

$$\omega_r = \sqrt{\frac{k}{m} - \frac{b^2}{2m^2}}$$

input	meaning	unit
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `rad/s`
- **File:** `amplitude_resonance_frequency__wr.onnx`
- **Applies when:** driven

```
inputs {'k': 2.0, 'm': 2.1, 'b': 2.2}
-> 0.635317 rad/s   (ran on amplitude_resonance_frequency__wr.onnx; matches the Python engine)
```

Acceleration of an Atwood machine with a massive pulley — solve for moment of inertia of the pulley — `atwood_massive_pulley.Ir`

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

input	meaning	unit
`a`	acceleration of each mass	`m/s^2`
`m1`	heavier mass	`kg`
`m2`	lighter mass	`kg`
`g`	acceleration	`m/s^2`
`R`	radius of the pulley	`m`

- **Output unit:** `kg*m^2`
- **File:** `atwood_massive_pulley__Ir.onnx`
- **Applies when:** atwood

Acceleration of an Atwood machine with a massive pulley — solve for radius of the pulley — `atwood_massive_pulley.R`

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

input	meaning	unit
`a`	acceleration of each mass	`m/s^2`
`m1`	heavier mass	`kg`
`m2`	lighter mass	`kg`
`g`	acceleration	`m/s^2`
`Ir`	moment of inertia of the pulley	`kg*m^2`

- **Output unit:** `m`
- **File:** `atwood_massive_pulley__R.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** atwood

Acceleration of an Atwood machine with a massive pulley — solve for acceleration of each mass — `atwood_massive_pulley.a`

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

input	meaning	unit
`m1`	heavier mass	`kg`
`m2`	lighter mass	`kg`
`g`	acceleration	`m/s^2`
`Ir`	moment of inertia of the pulley	`kg*m^2`
`R`	radius of the pulley	`m`

- **Output unit:** `m/s^2`
- **File:** `atwood_massive_pulley__a.onnx`
- **Applies when:** atwood

Acceleration of an Atwood machine with a massive pulley — solve for gravity — `atwood_massive_pulley.g`

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

input	meaning	unit
`a`	acceleration of each mass	`m/s^2`

input	meaning	unit
`m1`	heavier mass	`kg`
`m2`	lighter mass	`kg`
`Ir`	moment of inertia of the pulley	`kg*m^2`
`R`	radius of the pulley	`m`

- **Output unit:** `m/s^2`
- **File:** `atwood_massive_pulley__g.onnx`
- **Applies when:** atwood

Acceleration of an Atwood machine with a massive pulley — solve for heavier mass — `atwood_massive_pulley.m1`

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

input	meaning	unit
`a`	acceleration of each mass	`m/s^2`
`m2`	lighter mass	`kg`
`g`	acceleration	`m/s^2`
`Ir`	moment of inertia of the pulley	`kg*m^2`
`R`	radius of the pulley	`m`

- **Output unit:** `kg`
- **File:** `atwood_massive_pulley__m1.onnx`
- **Applies when:** atwood

```
inputs {'a': 2.0, 'm2': 2.1, 'g': 2.2, 'Ir': 2.3, 'R': 2.4}
-> 48.0931 kg   (ran on atwood_massive_pulley__m1.onnx; matches the Python engine)
```

Acceleration of an Atwood machine with a massive pulley — solve for lighter mass — `atwood_massive_pulley.m2`

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

input	meaning	unit
`a`	acceleration of each mass	`m/s^2`
`m1`	heavier mass	`kg`
`g`	acceleration	`m/s^2`
`Ir`	moment of inertia of the pulley	`kg*m^2`
`R`	radius of the pulley	`m`

- **Output unit:** `kg`

- **File:** `atwood_massive_pulley__m2.onnx`
- **Applies when:** atwood

```
inputs {'a': 5.0, 'm1': 5.1, 'g': 5.2, 'Ir': 5.3, 'R': 5.4}
-> 0.010904 kg   (ran on atwood_massive_pulley__m2.onnx; matches the Python engine)
```

Off-axis equilibrium of a bead on a rotating hoop — solve for radius of the hoop — `bead_hoop_equilibrium.R`

$$\theta = \arccos\left(\frac{g}{\omega^2 R}\right)$$

input	meaning	unit
`theta`	equilibrium angle from the bottom	`rad`
`g`	acceleration	`m/s^2`
`w`	angular velocity of the hoop	`rad/s`

- **Output unit:** `m`
- **File:** `bead_hoop_equilibrium__R.onnx`
- **Applies when:** hoop

```
inputs {'theta': 0.5, 'g': 0.6, 'w': 0.7}
-> 1.3953 m   (ran on bead_hoop_equilibrium__R.onnx; matches the Python engine)
```

Off-axis equilibrium of a bead on a rotating hoop — solve for gravity — `bead_hoop_equilibrium.g`

$$\theta = \arccos\left(\frac{g}{\omega^2 R}\right)$$

input	meaning	unit
`theta`	equilibrium angle from the bottom	`rad`
`w`	angular velocity of the hoop	`rad/s`
`R`	radius of the hoop	`m`

- **Output unit:** `m/s^2`
- **File:** `bead_hoop_equilibrium__g.onnx`
- **Applies when:** hoop

```
inputs {'theta': 0.5, 'w': 0.6, 'R': 0.7}
-> 0.221151 m/s^2   (ran on bead_hoop_equilibrium__g.onnx; matches the Python engine)
```

Off-axis equilibrium of a bead on a rotating hoop — solve for equilibrium angle from the bottom — `bead_hoop_equilibrium.theta`

$$\theta = \arccos\left(\frac{g}{\omega^2 R}\right)$$

input	meaning	unit
`g`	acceleration	`m/s^2`
`w`	angular velocity of the hoop	`rad/s`
`R`	radius of the hoop	`m`

- **Output unit:** `rad`
- **File:** `bead_hoop_equilibrium__theta.onnx`
- **Applies when:** hoop

```
inputs {'g': 2.0, 'w': 2.1, 'R': 2.2}
-> 1.36316 rad   (ran on bead_hoop_equilibrium__theta.onnx; matches the Python engine)
```

Off-axis equilibrium of a bead on a rotating hoop — solve for angular velocity of the hoop — `bead_hoop_equilibrium.w`

$$\theta = \arccos\left(\frac{g}{\omega^2 R}\right)$$

input	meaning	unit
`theta`	equilibrium angle from the bottom	`rad`
`g`	acceleration	`m/s^2`
`R`	radius of the hoop	`m`

- **Output unit:** `rad/s`
- **File:** `bead_hoop_equilibrium__w.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** hoop

```
inputs {'theta': 0.5, 'g': 0.6, 'R': 0.7}
-> -0.988286, 0.988286 rad/s   (ran on bead_hoop_equilibrium__w.onnx; matches the Python engine)
```

Higher normal-mode frequency of two coupled oscillators — solve for each outer spring's constant — `coupled_fast_mode.k`

$$\omega_2 = \sqrt{\frac{k + 2k_c}{m}}$$

input	meaning	unit
`w2`	higher normal-mode angular frequency	`rad/s`

input	meaning	unit
`kc`	coupling spring constant	`N/m`
`m`	each mass	`kg`

- **Output unit:** `N/m`
- **File:** `coupled_fast_mode__k.onnx`
- **Applies when:** coupled

```
inputs {'w2': 2.0, 'kc': 2.1, 'm': 2.2}
-> 4.6 N/m   (ran on coupled_fast_mode__k.onnx; matches the Python engine)
```

Higher normal-mode frequency of two coupled oscillators — solve for coupling spring constant — `coupled_fast_mode.kc`

$$\omega_2 = \sqrt{\frac{k + 2k_c}{m}}$$

input	meaning	unit
`w2`	higher normal-mode angular frequency	`rad/s`
`k`	each outer spring's constant	`N/m`
`m`	each mass	`kg`

- **Output unit:** `N/m`
- **File:** `coupled_fast_mode__kc.onnx`
- **Applies when:** coupled

```
inputs {'w2': 2.0, 'k': 2.1, 'm': 2.2}
-> 3.35 N/m   (ran on coupled_fast_mode__kc.onnx; matches the Python engine)
```

Higher normal-mode frequency of two coupled oscillators — solve for each mass — `coupled_fast_mode.m`

$$\omega_2 = \sqrt{\frac{k + 2k_c}{m}}$$

input	meaning	unit
`w2`	higher normal-mode angular frequency	`rad/s`
`k`	each outer spring's constant	`N/m`
`kc`	coupling spring constant	`N/m`

- **Output unit:** `kg`
- **File:** `coupled_fast_mode__m.onnx`
- **Applies when:** coupled

```
inputs {'w2': 2.0, 'k': 2.1, 'kc': 2.2}
-> 1.625 kg   (ran on coupled_fast_mode__m.onnx; matches the Python engine)
```

Higher normal-mode frequency of two coupled oscillators — solve for higher normal-mode angular frequency — `coupled_fast_mode.w2`

$$\omega_2 = \sqrt{\frac{k + 2k_c}{m}}$$

input	meaning	unit
`k`	each outer spring's constant	`N/m`
`kc`	coupling spring constant	`N/m`
`m`	each mass	`kg`

- **Output unit:** `rad/s`
- **File:** `coupled_fast_mode__w2.onnx`
- **Applies when:** coupled

```
inputs {'k': 2.0, 'kc': 2.1, 'm': 2.2}
-> 1.67874 rad/s   (ran on coupled_fast_mode__w2.onnx; matches the Python engine)
```

Angular frequency of a damped oscillator — solve for viscous damping coefficient — `damped_angular_frequency.b`

$$\omega_d = \sqrt{\frac{k}{m} - \frac{b^2}{4m^2}}$$

input	meaning	unit
`wd`	angular frequency of the damped oscillation	`rad/s`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`

- **Output unit:** `kg/s`
- **File:** `damped_angular_frequency__b.onnx`
- **Applies when:** damped

```
inputs {'wd': 0.5, 'k': 0.6, 'm': 0.7}
-> 1.09087 kg/s   (ran on damped_angular_frequency__b.onnx; matches the Python engine)
```

Angular frequency of a damped oscillator — solve for spring constant — `damped_angular_frequency.k`

$$\omega_d = \sqrt{\frac{k}{m} - \frac{b^2}{4m^2}}$$

input	meaning	unit
`wd`	angular frequency of the damped oscillation	`rad/s`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `N/m`
- **File:** `damped_angular_frequency__k.onnx`
- **Applies when:** damped

```
inputs {'wd': 2.0, 'm': 2.1, 'b': 2.2}
-> 8.97619 N/m   (ran on damped_angular_frequency__k.onnx; matches the Python engine)
```

Angular frequency of a damped oscillator — solve for oscillating mass — `damped_angular_frequency.m`

$$\omega_d = \sqrt{\frac{k}{m} - \frac{b^2}{4m^2}}$$

input	meaning	unit
`wd`	angular frequency of the damped oscillation	`rad/s`
`k`	spring constant	`N/m`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `kg`
- **File:** `damped_angular_frequency__m.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** damped

Angular frequency of a damped oscillator — solve for angular frequency of the damped oscillation — `damped_angular_frequency.wd`

$$\omega_d = \sqrt{\frac{k}{m} - \frac{b^2}{4m^2}}$$

input	meaning	unit
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `rad/s`
- **File:** `damped_angular_frequency__wd.onnx`
- **Applies when:** damped

```
inputs {'k': 2.0, 'm': 2.1, 'b': 2.2}
-> 0.82341 rad/s   (ran on damped_angular_frequency__wd.onnx; matches the Python engine)
```

Damping ratio — solve for viscous damping coefficient — `damping_ratio.b`

$$\zeta = \frac{b}{2\sqrt{km}}$$

input	meaning	unit
`zeta`	damping ratio	`1`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`

- **Output unit:** `kg/s`
- **File:** `damping_ratio__b.onnx`
- **Applies when:** damped

```
inputs {'zeta': 2.0, 'k': 2.1, 'm': 2.2}
-> 8.59767 kg/s   (ran on damping_ratio__b.onnx; matches the Python engine)
```

Damping ratio — solve for spring constant — `damping_ratio.k`

$$\zeta = \frac{b}{2\sqrt{km}}$$

input	meaning	unit
`zeta`	damping ratio	`1`
`b`	viscous damping coefficient	`kg/s`
`m`	oscillating mass	`kg`

- **Output unit:** `N/m`
- **File:** `damping_ratio__k.onnx`
- **Applies when:** damped

```
inputs {'zeta': 2.0, 'b': 2.1, 'm': 2.2}
-> 0.125284 N/m   (ran on damping_ratio__k.onnx; matches the Python engine)
```

Damping ratio — solve for oscillating mass — `damping_ratio.m`

$$\zeta = \frac{b}{2\sqrt{km}}$$

input	meaning	unit
`zeta`	damping ratio	`1`
`b`	viscous damping coefficient	`kg/s`
`k`	spring constant	`N/m`

- **Output unit:** `kg`

- **File:** `damping_ratio__m.onnx`
- **Applies when:** damped

```
inputs {'zeta': 2.0, 'b': 2.1, 'k': 2.2}
-> 0.125284 kg   (ran on damping_ratio__m.onnx; matches the Python engine)
```

Damping ratio — solve for damping ratio — `damping_ratio.zeta`

$$\zeta = \frac{b}{2\sqrt{km}}$$

input	meaning	unit
`b`	viscous damping coefficient	`kg/s`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`

- **Output unit:** `1`
- **File:** `damping_ratio__zeta.onnx`
- **Applies when:** damped

```
inputs {'b': 2.0, 'k': 2.1, 'm': 2.2}
-> 0.465242 1   (ran on damping_ratio__zeta.onnx; matches the Python engine)
```

Higher normal mode of a double pendulum — solve for length of each arm — `double_pendulum_fast_mode.L`

$$\omega = \sqrt{\frac{\left(2+\sqrt{2}\right)g}{L}}$$

input	meaning	unit
`w`	higher normal-mode angular frequency	`rad/s`
`g`	acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `double_pendulum_fast_mode__L.onnx`
- **Applies when:** double_pendulum

```
inputs {'w': 2.0, 'g': 2.1}
-> 1.79246 m   (ran on double_pendulum_fast_mode__L.onnx; matches the Python engine)
```

Higher normal mode of a double pendulum — solve for gravity — `double_pendulum_fast_mode.g`

$$\omega = \sqrt{\frac{\left(2+\sqrt{2}\right)g}{L}}$$

input	meaning	unit
`w`	higher normal-mode angular frequency	`rad/s`
`L`	length of each arm	`m`

- **Output unit:** `m/s^2`
- **File:** `double_pendulum_fast_mode__g.onnx`
- **Applies when:** double_pendulum

```
inputs {'w': 2.0, 'L': 2.1}
-> 2.4603 m/s^2   (ran on double_pendulum_fast_mode__g.onnx; matches the Python engine)
```

Higher normal mode of a double pendulum — solve for higher normal-mode angular frequency — `double_pendulum_fast_mode.w`

$$\omega = \sqrt{\frac{\left(2+\sqrt{2}\right)g}{L}}$$

input	meaning	unit
`g`	acceleration	`m/s^2`
`L`	length of each arm	`m`

- **Output unit:** `rad/s`
- **File:** `double_pendulum_fast_mode__w.onnx`
- **Applies when:** double_pendulum

```
inputs {'g': 2.0, 'L': 2.1}
-> 1.80323 rad/s   (ran on double_pendulum_fast_mode__w.onnx; matches the Python engine)
```

Lower normal mode of a double pendulum — solve for length of each arm — `double_pendulum_slow_mode.L`

$$\omega = \sqrt{\frac{\left(2-\sqrt{2}\right)g}{L}}$$

input	meaning	unit
`w`	lower normal-mode angular frequency	`rad/s`

input	meaning	unit
`g`	acceleration	`m/s^2`

- **Output unit:** `m`
- **File:** `double_pendulum_slow_mode__L.onnx`
- **Applies when:** double_pendulum

```
inputs {'w': 2.0, 'g': 2.1}
-> 0.307538 m   (ran on double_pendulum_slow_mode__L.onnx; matches the Python engine)
```

Lower normal mode of a double pendulum — solve for gravity — `double_pendulum_slow_mode.g`

$$\omega = \sqrt{\frac{\left(2-\sqrt{2}\right)g}{L}}$$

input	meaning	unit
`w`	lower normal-mode angular frequency	`rad/s`
`L`	length of each arm	`m`

- **Output unit:** `m/s^2`
- **File:** `double_pendulum_slow_mode__g.onnx`
- **Applies when:** double_pendulum

```
inputs {'w': 2.0, 'L': 2.1}
-> 14.3397 m/s^2   (ran on double_pendulum_slow_mode__g.onnx; matches the Python engine)
```

Lower normal mode of a double pendulum — solve for lower normal-mode angular frequency — `double_pendulum_slow_mode.w`

$$\omega = \sqrt{\frac{\left(2-\sqrt{2}\right)g}{L}}$$

input	meaning	unit
`g`	acceleration	`m/s^2`
`L`	length of each arm	`m`

- **Output unit:** `rad/s`
- **File:** `double_pendulum_slow_mode__w.onnx`
- **Applies when:** double_pendulum

```
inputs {'g': 2.0, 'L': 2.1}
-> 0.746922 rad/s   (ran on double_pendulum_slow_mode__w.onnx; matches the Python engine)
```

Steady-state amplitude of a driven damped oscillator — solve for steady-state amplitude — `driven_amplitude.A`

$$A = \frac{F_0}{\sqrt{(k - m\omega^2)^2 + (b\omega)^2}}$$

input	meaning	unit
`F0`	driving force amplitude	`N`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`
`w`	driving angular frequency	`rad/s`

- **Output unit:** `m`
- **File:** `driven_amplitude__A.onnx`
- **Applies when:** driven

```
inputs {'F0': 2.0, 'k': 2.1, 'm': 2.2, 'b': 2.3, 'w': 2.4}
-> 0.167696 m   (ran on driven_amplitude__A.onnx; matches the Python engine)
```

Steady-state amplitude of a driven damped oscillator — solve for driving force amplitude — `driven_amplitude.F0`

$$A = \frac{F_0}{\sqrt{(k - m\omega^2)^2 + (b\omega)^2}}$$

input	meaning	unit
`A`	steady-state amplitude	`m`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`
`w`	driving angular frequency	`rad/s`

- **Output unit:** `N`
- **File:** `driven_amplitude__F0.onnx`
- **Applies when:** driven

```
inputs {'A': 2.0, 'k': 2.1, 'm': 2.2, 'b': 2.3, 'w': 2.4}
-> 23.8527 N   (ran on driven_amplitude__F0.onnx; matches the Python engine)
```

Steady-state amplitude of a driven damped oscillator — solve for viscous damping coefficient — `driven_amplitude.b`

$$A = \frac{F_0}{\sqrt{\left(k - m\omega^2\right)^2 + \left(b\omega\right)^2}}$$

input	meaning	unit
`A`	steady-state amplitude	`m`
`F0`	driving force amplitude	`N`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`
`w`	driving angular frequency	`rad/s`

- **Output unit:** `kg/s`
- **File:** `driven_amplitude__b.onnx`
- **Applies when:** driven

```
inputs {'A': 0.5, 'F0': 0.6, 'k': 0.7, 'm': 0.8, 'w': 0.9}
-> 1.33208 kg/s   (ran on driven_amplitude__b.onnx; matches the Python engine)
```

Steady-state amplitude of a driven damped oscillator — solve for spring constant — `driven_amplitude.k`

$$A = \frac{F_0}{\sqrt{\left(k - m\omega^2\right)^2 + \left(b\omega\right)^2}}$$

input	meaning	unit
`A`	steady-state amplitude	`m`
`F0`	driving force amplitude	`N`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`
`w`	driving angular frequency	`rad/s`

- **Output unit:** `N/m`
- **File:** `driven_amplitude__k.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** driven

```
inputs {'A': 0.5, 'F0': 0.6, 'm': 0.7, 'b': 0.8, 'w': 0.9}
-> -0.393, 1.527 N/m   (ran on driven_amplitude__k.onnx; matches the Python engine)
```

Steady-state amplitude of a driven damped oscillator — solve for oscillating mass — `driven_amplitude.m`

$$A = \frac{F_0}{\sqrt{(k - m\omega^2)^2 + (b\omega)^2}}$$

input	meaning	unit
`A`	steady-state amplitude	`m`
`F0`	driving force amplitude	`N`
`k`	spring constant	`N/m`
`b`	viscous damping coefficient	`kg/s`
`w`	driving angular frequency	`rad/s`

- **Output unit:** `kg`
- **File:** `driven_amplitude__m.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** driven

```
inputs {'A': 0.5, 'F0': 0.6, 'k': 0.7, 'b': 0.8, 'w': 0.9}
-> -0.320988, 2.04938 kg   (ran on driven_amplitude__m.onnx; matches the Python engine)
```

Steady-state amplitude of a driven damped oscillator — solve for driving angular frequency — `driven_amplitude.w`

$$A = \frac{F_0}{\sqrt{(k - m\omega^2)^2 + (b\omega)^2}}$$

input	meaning	unit
`A`	steady-state amplitude	`m`
`F0`	driving force amplitude	`N`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `rad/s`
- **File:** `driven_amplitude__w.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** driven

```
inputs {'A': 0.5, 'F0': 0.6, 'k': 0.7, 'm': 0.8, 'b': 0.9}
-> nan, 1.21835 rad/s   (ran on driven_amplitude__w.onnx; matches the Python engine)
```

Final velocity of the first body in a one-dimensional elastic collision — solve for mass of the first body — `elastic_collision_first_final.m1`

$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2v_2}{m_1 + m_2}$$

input	meaning	unit
`v1f`	final velocity of the first body	`m/s`
`m2`	mass of the second body	`kg`
`v1`	initial velocity of the first body	`m/s`
`v2`	initial velocity of the second body	`m/s`

- **Output unit:** `kg`
- **File:** `elastic_collision_first_final__m1.onnx`
- **Applies when:** elastic_collision

Final velocity of the first body in a one-dimensional elastic collision — solve for mass of the second body — `elastic_collision_first_final.m2`

$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2v_2}{m_1 + m_2}$$

input	meaning	unit
`v1f`	final velocity of the first body	`m/s`
`m1`	mass of the first body	`kg`
`v1`	initial velocity of the first body	`m/s`
`v2`	initial velocity of the second body	`m/s`

- **Output unit:** `kg`
- **File:** `elastic_collision_first_final__m2.onnx`
- **Applies when:** elastic_collision

Final velocity of the first body in a one-dimensional elastic collision — solve for initial velocity of the first body — `elastic_collision_first_final.v1`

$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2v_2}{m_1 + m_2}$$

input	meaning	unit
`v1f`	final velocity of the first body	`m/s`
`m1`	mass of the first body	`kg`
`m2`	mass of the second body	`kg`
`v2`	initial velocity of the second body	`m/s`

- **Output unit:** `m/s`
- **File:** `elastic_collision_first_final__v1.onnx`
- **Applies when:** elastic_collision

```
inputs {'v1f': 2.0, 'm1': 2.1, 'm2': 2.2, 'v2': 2.3}
-> 15.2 m/s   (ran on elastic_collision_first_final__v1.onnx; matches the Python engine)
```

Final velocity of the first body in a one-dimensional elastic collision — solve for final velocity of the first body — `elastic_collision_first_final.v1f`

$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2v_2}{m_1 + m_2}$$

input	meaning	unit
`m1`	mass of the first body	`kg`
`m2`	mass of the second body	`kg`
`v1`	initial velocity of the first body	`m/s`
`v2`	initial velocity of the second body	`m/s`

- **Output unit:** `m/s`
- **File:** `elastic_collision_first_final__v1f.onnx`
- **Applies when:** elastic_collision

```
inputs {'m1': 2.0, 'm2': 2.1, 'v1': 2.2, 'v2': 2.3}
-> 2.30244 m/s   (ran on elastic_collision_first_final__v1f.onnx; matches the Python engine)
```

Final velocity of the first body in a one-dimensional elastic collision — solve for initial velocity of the second body — `elastic_collision_first_final.v2`

$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2v_2}{m_1 + m_2}$$

input	meaning	unit
`v1f`	final velocity of the first body	`m/s`
`m1`	mass of the first body	`kg`
`m2`	mass of the second body	`kg`
`v1`	initial velocity of the first body	`m/s`

- **Output unit:** `m/s`
- **File:** `elastic_collision_first_final__v2.onnx`
- **Applies when:** elastic_collision

```
inputs {'v1f': 2.0, 'm1': 2.1, 'm2': 2.2, 'v1': 2.3}
-> 2.00682 m/s   (ran on elastic_collision_first_final__v2.onnx; matches the Python engine)
```

Final velocity of the second body in a one-dimensional elastic collision — solve for mass of the first body — `elastic_collision_second_final.m1`

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$

input	meaning	unit
`v2f`	final velocity of the second body	`m/s`
`m2`	mass of the second body	`kg`
`v1`	initial velocity of the first body	`m/s`
`v2`	initial velocity of the second body	`m/s`

- **Output unit:** `kg`
- **File:** `elastic_collision_second_final__m1.onnx`
- **Applies when:** elastic_collision

Final velocity of the second body in a one-dimensional elastic collision — solve for mass of the second body — `elastic_collision_second_final.m2`

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$

input	meaning	unit
`v2f`	final velocity of the second body	`m/s`
`m1`	mass of the first body	`kg`
`v1`	initial velocity of the first body	`m/s`
`v2`	initial velocity of the second body	`m/s`

- **Output unit:** `kg`
- **File:** `elastic_collision_second_final__m2.onnx`
- **Applies when:** elastic_collision

Final velocity of the second body in a one-dimensional elastic collision — solve for initial velocity of the first body — `elastic_collision_second_final.v1`

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$

input	meaning	unit
`v2f`	final velocity of the second body	`m/s`
`m1`	mass of the first body	`kg`
`m2`	mass of the second body	`kg`
`v2`	initial velocity of the second body	`m/s`

- **Output unit:** `m/s`
- **File:** `elastic_collision_second_final__v1.onnx`
- **Applies when:** elastic_collision

```
inputs {'v2f': 2.0, 'm1': 2.1, 'm2': 2.2, 'v2': 2.3}
-> 1.99286 m/s   (ran on elastic_collision_second_final__v1.onnx; matches the Python engine)
```

Final velocity of the second body in a one-dimensional elastic collision — solve for initial velocity of the second body — `elastic_collision_second_final.v2`

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$

input	meaning	unit
`v2f`	final velocity of the second body	`m/s`
`m1`	mass of the first body	`kg`
`m2`	mass of the second body	`kg`
`v1`	initial velocity of the first body	`m/s`

- **Output unit:** `m/s`
- **File:** `elastic_collision_second_final__v2.onnx`
- **Applies when:** elastic_collision

```
inputs {'v2f': 2.0, 'm1': 2.1, 'm2': 2.2, 'v1': 2.3}
-> -10.6 m/s   (ran on elastic_collision_second_final__v2.onnx; matches the Python engine)
```

Final velocity of the second body in a one-dimensional elastic collision — solve for final velocity of the second body — `elastic_collision_second_final.v2f`

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$

input	meaning	unit
`m1`	mass of the first body	`kg`
`m2`	mass of the second body	`kg`
`v1`	initial velocity of the first body	`m/s`
`v2`	initial velocity of the second body	`m/s`

- **Output unit:** `m/s`
- **File:** `elastic_collision_second_final__v2f.onnx`
- **Applies when:** elastic_collision

```
inputs {'m1': 2.0, 'm2': 2.1, 'v1': 2.2, 'v2': 2.3}
-> 2.20244 m/s   (ran on elastic_collision_second_final__v2f.onnx; matches the Python engine)
```

Escape speed — solve for constant — `escape_speed.G`

$$v = \sqrt{\frac{2GM}{r}}$$

input	meaning	unit
`v`	escape speed	`m/s`
`M`	mass of the body	`kg`
`r`	distance from its center	`m`

- **Output unit:** `N*m^2/kg^2`
- **File:** `escape_speed__G.onnx`
- **Applies when:** escape

```
inputs {'v': 2.0, 'M': 2.1, 'r': 2.2}
-> 2.09524 N*m^2/kg^2   (ran on escape_speed__G.onnx; matches the Python engine)
```

Escape speed — solve for mass of the body — `escape_speed.M`

$$v = \sqrt{\frac{2GM}{r}}$$

input	meaning	unit
`v`	escape speed	`m/s`
`G`	gravitational constant	`N*m^2/kg^2`
`r`	distance from its center	`m`

- **Output unit:** `kg`
- **File:** `escape_speed__M.onnx`
- **Applies when:** escape

```
inputs {'v': 2.0, 'G': 2.1, 'r': 2.2}
-> 2.09524 kg   (ran on escape_speed__M.onnx; matches the Python engine)
```

Escape speed — solve for distance from its center — `escape_speed.r`

$$v = \sqrt{\frac{2GM}{r}}$$

input	meaning	unit
`v`	escape speed	`m/s`
`G`	gravitational constant	`N*m^2/kg^2`
`M`	mass of the body	`kg`

- **Output unit:** `m`

- **File:** `escape_speed__r.onnx`
- **Applies when:** escape

```
inputs {'v': 2.0, 'G': 2.1, 'M': 2.2}
-> 2.31 m   (ran on escape_speed__r.onnx; matches the Python engine)
```

Escape speed — solve for escape speed — `escape_speed.v`

$$v = \sqrt{\frac{2GM}{r}}$$

input	meaning	unit
`G`	gravitational constant	`N*m^2/kg^2`
`M`	mass of the body	`kg`
`r`	distance from its center	`m`

- **Output unit:** `m/s`
- **File:** `escape_speed__v.onnx`
- **Applies when:** escape

```
inputs {'G': 2.0, 'M': 2.1, 'r': 2.2}
-> 1.95402 m/s   (ran on escape_speed__v.onnx; matches the Python engine)
```

Precession period of a Foucault pendulum — solve for time for the swing plane to turn once — `foucault_precession_period.T`

$$T = \frac{2\pi}{\Omega \sin(\phi)}$$

input	meaning	unit
`W`	angular frequency	`rad/s`
`phi`	latitude	`rad`

- **Output unit:** `s`
- **File:** `foucault_precession_period__T.onnx`
- **Applies when:** foucault

```
inputs {'W': 2.0, 'phi': 2.1}
-> 3.63943 s   (ran on foucault_precession_period__T.onnx; matches the Python engine)
```

Precession period of a Foucault pendulum — solve for rotation — `foucault_precession_period.W`

$$T = \frac{2\pi}{\Omega \sin(\phi)}$$

input	meaning	unit
`T`	time for the swing plane to turn once	`s`
`phi`	latitude	`rad`

- **Output unit:** `rad/s`
- **File:** `foucault_precession_period__W.onnx`
- **Applies when:** foucault

```
inputs {'T': 2.0, 'phi': 2.1}
-> 3.63943 rad/s   (ran on foucault_precession_period__W.onnx; matches the Python engine)
```

Precession period of a Foucault pendulum — solve for latitude — `foucault_precession_period.phi`

$$T = \frac{2\pi}{\Omega \sin(\phi)}$$

input	meaning	unit
`T`	time for the swing plane to turn once	`s`
`W`	angular frequency	`rad/s`

- **Output unit:** `rad`
- **File:** `foucault_precession_period__phi.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** foucault

Moment of inertia of a thin spherical shell — solve for moment of inertia about the central axis — `inertia_hollow_sphere.Ir`

$$I_r = \frac{2mR^2}{3}$$

input	meaning	unit
`m`	mass	`kg`
`R`	radius	`m`

- **Output unit:** `kg*m^2`
- **File:** `inertia_hollow_sphere__Ir.onnx`
- **Applies when:** hollow_sphere

```
inputs {'m': 2.0, 'R': 2.1}
-> 5.88 kg*m^2   (ran on inertia_hollow_sphere__Ir.onnx; matches the Python engine)
```

Moment of inertia of a thin spherical shell — solve for radius — `inertia_hollow_sphere.R`

$$I_r = \frac{2mR^2}{3}$$

input	meaning	unit
`Ir`	moment of inertia about the central axis	`kg*m^2`
`m`	mass	`kg`

- **Output unit:** `m`
- **File:** `inertia_hollow_sphere__R.onnx`
- **Applies when:** hollow_sphere

```
inputs {'Ir': 2.0, 'm': 2.1}
-> 1.19523 m   (ran on inertia_hollow_sphere__R.onnx; matches the Python engine)
```

Moment of inertia of a thin spherical shell — solve for mass — `inertia_hollow_sphere.m`

$$I_r = \frac{2mR^2}{3}$$

input	meaning	unit
`Ir`	moment of inertia about the central axis	`kg*m^2`
`R`	radius	`m`

- **Output unit:** `kg`
- **File:** `inertia_hollow_sphere__m.onnx`
- **Applies when:** hollow_sphere

```
inputs {'Ir': 2.0, 'R': 2.1}
-> 0.680272 kg   (ran on inertia_hollow_sphere__m.onnx; matches the Python engine)
```

Moment of inertia of a thin rod about its center — solve for moment of inertia about its center — `inertia_rod_center.Ir`

$$I_r = \frac{mL_r^2}{12}$$

input	meaning	unit
`m`	mass of the rod	`kg`
`Lr`	length of the rod	`m`

- **Output unit:** `kg*m^2`
- **File:** `inertia_rod_center__Ir.onnx`
- **Applies when:** rod; about_center

```
inputs {'m': 2.0, 'Lr': 2.1}
-> 0.735 kg*m^2   (ran on inertia_rod_center__Ir.onnx; matches the Python engine)
```

Moment of inertia of a thin rod about its center — solve for length of the rod — `inertia_rod_center.Lr`

$$I_r = \frac{mL_r^2}{12}$$

input	meaning	unit
`Ir`	moment of inertia about its center	`kg*m^2`
`m`	mass of the rod	`kg`

- **Output unit:** `m`
- **File:** `inertia_rod_center__Lr.onnx`
- **Applies when:** rod; about_center

```
inputs {'Ir': 2.0, 'm': 2.1}
-> 3.38062 m   (ran on inertia_rod_center__Lr.onnx; matches the Python engine)
```

Moment of inertia of a thin rod about its center — solve for mass of the rod — `inertia_rod_center.m`

$$I_r = \frac{mL_r^2}{12}$$

input	meaning	unit
`Ir`	moment of inertia about its center	`kg*m^2`
`Lr`	length of the rod	`m`

- **Output unit:** `kg`
- **File:** `inertia_rod_center__m.onnx`
- **Applies when:** rod; about_center

```
inputs {'Ir': 2.0, 'Lr': 2.1}
-> 5.44218 kg   (ran on inertia_rod_center__m.onnx; matches the Python engine)
```

Moment of inertia of a thin rod about one end — solve for moment of inertia about one end — `inertia_rod_end.Ir`

$$I_r = \frac{mL_r^2}{3}$$

input	meaning	unit
`m`	mass of the rod	`kg`

input	meaning	unit
`Lr`	length of the rod	`m`

- **Output unit:** `kg*m^2`
- **File:** `inertia_rod_end__Ir.onnx`
- **Applies when:** rod; about_end

```
inputs {'m': 2.0, 'Lr': 2.1}
-> 2.94 kg*m^2   (ran on inertia_rod_end__Ir.onnx; matches the Python engine)
```

Moment of inertia of a thin rod about one end — solve for length of the rod — `inertia_rod_end.Lr`

$$I_r = \frac{mL_r^2}{3}$$

input	meaning	unit
`Ir`	moment of inertia about one end	`kg*m^2`
`m`	mass of the rod	`kg`

- **Output unit:** `m`
- **File:** `inertia_rod_end__Lr.onnx`
- **Applies when:** rod; about_end

```
inputs {'Ir': 2.0, 'm': 2.1}
-> 1.69031 m   (ran on inertia_rod_end__Lr.onnx; matches the Python engine)
```

Moment of inertia of a thin rod about one end — solve for mass of the rod — `inertia_rod_end.m`

$$I_r = \frac{mL_r^2}{3}$$

input	meaning	unit
`Ir`	moment of inertia about one end	`kg*m^2`
`Lr`	length of the rod	`m`

- **Output unit:** `kg`
- **File:** `inertia_rod_end__m.onnx`
- **Applies when:** rod; about_end

```
inputs {'Ir': 2.0, 'Lr': 2.1}
-> 1.36054 kg   (ran on inertia_rod_end__m.onnx; matches the Python engine)
```

Moment of inertia of a solid disk about its axis — solve for moment of inertia about the central axis — `inertia_solid_disk.Ir`

$$I_r = \frac{mR^2}{2}$$

input	meaning	unit
`m`	mass	`kg`
`R`	radius	`m`

- **Output unit:** `kg*m^2`
- **File:** `inertia_solid_disk__Ir.onnx`
- **Applies when:** solid_disk

```
inputs {'m': 2.0, 'R': 2.1}
-> 4.41 kg*m^2   (ran on inertia_solid_disk__Ir.onnx; matches the Python engine)
```

Moment of inertia of a solid disk about its axis — solve for radius — `inertia_solid_disk.R`

$$I_r = \frac{mR^2}{2}$$

input	meaning	unit
`Ir`	moment of inertia about the central axis	`kg*m^2`
`m`	mass	`kg`

- **Output unit:** `m`
- **File:** `inertia_solid_disk__R.onnx`
- **Applies when:** solid_disk

```
inputs {'Ir': 2.0, 'm': 2.1}
-> 1.38013 m   (ran on inertia_solid_disk__R.onnx; matches the Python engine)
```

Moment of inertia of a solid disk about its axis — solve for mass — `inertia_solid_disk.m`

$$I_r = \frac{mR^2}{2}$$

input	meaning	unit
`Ir`	moment of inertia about the central axis	`kg*m^2`
`R`	radius	`m`

- **Output unit:** `kg`
- **File:** `inertia_solid_disk__m.onnx`
- **Applies when:** solid_disk

```
inputs {'Ir': 2.0, 'R': 2.1}
-> 0.907029 kg   (ran on inertia_solid_disk__m.onnx; matches the Python engine)
```

Moment of inertia of a solid sphere — solve for moment of inertia about the central axis — `inertia_solid_sphere.Ir`

$$I_r = \frac{2mR^2}{5}$$

input	meaning	unit
`m`	mass	`kg`
`R`	radius	`m`

- **Output unit:** `kg*m^2`
- **File:** `inertia_solid_sphere__Ir.onnx`
- **Applies when:** solid_sphere

```
inputs {'m': 2.0, 'R': 2.1}
-> 3.528 kg*m^2   (ran on inertia_solid_sphere__Ir.onnx; matches the Python engine)
```

Moment of inertia of a solid sphere — solve for radius — `inertia_solid_sphere.R`

$$I_r = \frac{2mR^2}{5}$$

input	meaning	unit
`Ir`	moment of inertia about the central axis	`kg*m^2`
`m`	mass	`kg`

- **Output unit:** `m`
- **File:** `inertia_solid_sphere__R.onnx`
- **Applies when:** solid_sphere

```
inputs {'Ir': 2.0, 'm': 2.1}
-> 1.54303 m   (ran on inertia_solid_sphere__R.onnx; matches the Python engine)
```

Moment of inertia of a solid sphere — solve for mass — `inertia_solid_sphere.m`

$$I_r = \frac{2mR^2}{5}$$

input	meaning	unit
`Ir`	moment of inertia about the central axis	`kg*m^2`
`R`	radius	`m`

- **Output unit:** `kg`
- **File:** `inertia_solid_sphere__m.onnx`
- **Applies when:** solid_sphere

```
inputs {'Ir': 2.0, 'R': 2.1}
-> 1.13379 kg   (ran on inertia_solid_sphere__m.onnx; matches the Python engine)
```

Orbital period from the semi-major axis (Kepler's third law) — solve for constant — `kepler_period_semi_major.G`

$$T = 2\pi\sqrt{\frac{a^3}{GM}}$$

input	meaning	unit
`T`	orbital period	`s`
`a`	semi-major axis	`m`
`M`	mass of the central body	`kg`

- **Output unit:** `N*m^2/kg^2`
- **File:** `kepler_period_semi_major__G.onnx`
- **Applies when:** orbit

```
inputs {'T': 2.0, 'a': 2.1, 'M': 2.2}
-> 41.5465 N*m^2/kg^2   (ran on kepler_period_semi_major__G.onnx; matches the Python engine)
```

Orbital period from the semi-major axis (Kepler's third law) — solve for mass of the central body — `kepler_period_semi_major.M`

$$T = 2\pi\sqrt{\frac{a^3}{GM}}$$

input	meaning	unit
`T`	orbital period	`s`
`a`	semi-major axis	`m`
`G`	gravitational constant	`N*m^2/kg^2`

- **Output unit:** `kg`
- **File:** `kepler_period_semi_major__M.onnx`
- **Applies when:** orbit

```
inputs {'T': 2.0, 'a': 2.1, 'G': 2.2}
-> 41.5465 kg   (ran on kepler_period_semi_major__M.onnx; matches the Python engine)
```

Orbital period from the semi-major axis (Kepler's third law) — solve for orbital period — `kepler_period_semi_major.T`

$$T = 2\pi\sqrt{\frac{a^3}{GM}}$$

input	meaning	unit
`a`	semi-major axis	`m`
`G`	gravitational constant	`N*m^2/kg^2`
`M`	mass of the central body	`kg`

- **Output unit:** `s`
- **File:** `kepler_period_semi_major__T.onnx`
- **Applies when:** orbit

```
inputs {'a': 2.0, 'G': 2.1, 'M': 2.2}
-> 8.26806 s   (ran on kepler_period_semi_major__T.onnx; matches the Python engine)
```

Orbital period from the semi-major axis (Kepler's third law) — solve for semi-major axis — `kepler_period_semi_major.a`

$$T = 2\pi\sqrt{\frac{a^3}{GM}}$$

input	meaning	unit
`T`	orbital period	`s`
`G`	gravitational constant	`N*m^2/kg^2`
`M`	mass of the central body	`kg`

- **Output unit:** `m`
- **File:** `kepler_period_semi_major__a.onnx`
- **Applies when:** orbit

```
inputs {'T': 2.0, 'G': 2.1, 'M': 2.2}
-> 0.776451 m   (ran on kepler_period_semi_major__a.onnx; matches the Python engine)
```

Period of a physical pendulum — solve for moment of inertia about the pivot — `physical_pendulum_period.Ir`

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

input	meaning	unit
`T`	small-oscillation period	`s`

input	meaning	unit
`m`	mass of the body	`kg`
`g`	acceleration	`m/s^2`
`d`	distance from the pivot to the center of mass	`m`

- **Output unit:** `kg*m^2`
- **File:** `physical_pendulum_period__Ir.onnx`
- **Applies when:** physical_pendulum

```
inputs {'T': 2.0, 'm': 2.1, 'g': 2.2, 'd': 2.3}
-> 1.07664 kg*m^2   (ran on physical_pendulum_period__Ir.onnx; matches the Python engine)
```

Period of a physical pendulum — solve for small-oscillation period — `physical_pendulum_period.T`

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

input	meaning	unit
`Ir`	moment of inertia about the pivot	`kg*m^2`
`m`	mass of the body	`kg`
`g`	acceleration	`m/s^2`
`d`	distance from the pivot to the center of mass	`m`

- **Output unit:** `s`
- **File:** `physical_pendulum_period__T.onnx`
- **Applies when:** physical_pendulum

```
inputs {'Ir': 2.0, 'm': 2.1, 'g': 2.2, 'd': 2.3}
-> 2.7259 s   (ran on physical_pendulum_period__T.onnx; matches the Python engine)
```

Period of a physical pendulum — solve for distance from the pivot to the center of mass — `physical_pendulum_period.d`

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

input	meaning	unit
`T`	small-oscillation period	`s`
`Ir`	moment of inertia about the pivot	`kg*m^2`
`m`	mass of the body	`kg`
`g`	acceleration	`m/s^2`

- **Output unit:** `m`

- **File:** `physical_pendulum_period__d.onnx`
- **Applies when:** physical_pendulum

```
inputs {'T': 2.0, 'Ir': 2.1, 'm': 2.2, 'g': 2.3}
-> 4.09608 m   (ran on physical_pendulum_period__d.onnx; matches the Python engine)
```

Period of a physical pendulum — solve for gravity — `physical_pendulum_period.g`

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

input	meaning	unit
`T`	small-oscillation period	`s`
`Ir`	moment of inertia about the pivot	`kg*m^2`
`m`	mass of the body	`kg`
`d`	distance from the pivot to the center of mass	`m`

- **Output unit:** `m/s^2`
- **File:** `physical_pendulum_period__g.onnx`
- **Applies when:** physical_pendulum

```
inputs {'T': 2.0, 'Ir': 2.1, 'm': 2.2, 'd': 2.3}
-> 4.09608 m/s^2   (ran on physical_pendulum_period__g.onnx; matches the Python engine)
```

Period of a physical pendulum — solve for mass of the body — `physical_pendulum_period.m`

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

input	meaning	unit
`T`	small-oscillation period	`s`
`Ir`	moment of inertia about the pivot	`kg*m^2`
`g`	acceleration	`m/s^2`
`d`	distance from the pivot to the center of mass	`m`

- **Output unit:** `kg`
- **File:** `physical_pendulum_period__m.onnx`
- **Applies when:** physical_pendulum

```
inputs {'T': 2.0, 'Ir': 2.1, 'g': 2.2, 'd': 2.3}
-> 4.09608 kg   (ran on physical_pendulum_period__m.onnx; matches the Python engine)
```

Quality factor of a damped oscillator — solve for quality factor — `quality_factor.Qf`

$$Q_f = \frac{\sqrt{km}}{b}$$

input	meaning	unit
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `1`
- **File:** `quality_factor__Qf.onnx`
- **Applies when:** damped

```
inputs {'k': 2.0, 'm': 2.1, 'b': 2.2}
-> 0.931541 1   (ran on quality_factor__Qf.onnx; matches the Python engine)
```

Quality factor of a damped oscillator — solve for viscous damping coefficient — `quality_factor.b`

$$Q_f = \frac{\sqrt{km}}{b}$$

input	meaning	unit
`Qf`	quality factor	`1`
`k`	spring constant	`N/m`
`m`	oscillating mass	`kg`

- **Output unit:** `kg/s`
- **File:** `quality_factor__b.onnx`
- **Applies when:** damped

```
inputs {'Qf': 2.0, 'k': 2.1, 'm': 2.2}
-> 1.07471 kg/s   (ran on quality_factor__b.onnx; matches the Python engine)
```

Quality factor of a damped oscillator — solve for spring constant — `quality_factor.k`

$$Q_f = \frac{\sqrt{km}}{b}$$

input	meaning	unit
`Qf`	quality factor	`1`
`m`	oscillating mass	`kg`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `N/m`
- **File:** `quality_factor__k.onnx`
- **Applies when:** damped

```
inputs {'Qf': 2.0, 'm': 2.1, 'b': 2.2}
-> 9.21905 N/m   (ran on quality_factor__k.onnx; matches the Python engine)
```

Quality factor of a damped oscillator — solve for oscillating mass — `quality_factor.m`

$$Q_f = \frac{\sqrt{km}}{b}$$

input	meaning	unit
`Qf`	quality factor	`1`
`k`	spring constant	`N/m`
`b`	viscous damping coefficient	`kg/s`

- **Output unit:** `kg`
- **File:** `quality_factor__m.onnx`
- **Applies when:** damped

```
inputs {'Qf': 2.0, 'k': 2.1, 'b': 2.2}
-> 9.21905 kg   (ran on quality_factor__m.onnx; matches the Python engine)
```

Velocity change of a rocket (Tsiolkovsky) — solve for change in velocity — `rocket_delta_v.dv`

$$\Delta v = v_e \ln\left(\frac{m_0}{m_f}\right)$$

input	meaning	unit
`ve`	exhaust velocity	`m/s`
`m0`	initial mass	`kg`
`mf`	final mass	`kg`

- **Output unit:** `m/s`
- **File:** `rocket_delta_v__dv.onnx`
- **Applies when:** rocket

Velocity change of a rocket (Tsiolkovsky) — solve for initial mass — `rocket_delta_v.m0`

$$\Delta v = v_e \ln\left(\frac{m_0}{m_f}\right)$$

input	meaning	unit
`dv`	change in velocity	`m/s`

input	meaning	unit
`ve`	exhaust velocity	`m/s`
`mf`	final mass	`kg`

- **Output unit:** `kg`
- **File:** `rocket_delta_v__m0.onnx`
- **Applies when:** rocket

```
inputs {'dv': 2.0, 've': 2.1, 'mf': 2.2}
-> 5.70212 kg   (ran on rocket_delta_v__m0.onnx; matches the Python engine)
```

Velocity change of a rocket (Tsiolkovsky) — solve for final mass — `rocket_delta_v.mf`

$$\Delta v = v_e \ln\left(\frac{m_0}{m_f}\right)$$

input	meaning	unit
`dv`	change in velocity	`m/s`
`ve`	exhaust velocity	`m/s`
`m0`	initial mass	`kg`

- **Output unit:** `kg`
- **File:** `rocket_delta_v__mf.onnx`
- **Applies when:** rocket

```
inputs {'dv': 2.0, 've': 2.1, 'm0': 2.2}
-> 0.848807 kg   (ran on rocket_delta_v__mf.onnx; matches the Python engine)
```

Velocity change of a rocket (Tsiolkovsky) — solve for exhaust velocity — `rocket_delta_v.ve`

$$\Delta v = v_e \ln\left(\frac{m_0}{m_f}\right)$$

input	meaning	unit
`dv`	change in velocity	`m/s`
`m0`	initial mass	`kg`
`mf`	final mass	`kg`

- **Output unit:** `m/s`
- **File:** `rocket_delta_v__ve.onnx`
- **Applies when:** rocket

Acceleration of a body rolling without slipping down an incline — solve for moment of inertia about the center — `rolling_incline_acceleration.Ir`

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{mR^2}}$$

input	meaning	unit
`a`	acceleration of the center of mass along the incline	`m/s^2`
`g`	acceleration	`m/s^2`
`theta`	angle of the incline	`rad`
`m`	mass	`kg`
`R`	rolling radius	`m`

- **Output unit:** `kg*m^2`
- **File:** `rolling_incline_acceleration__Ir.onnx`
- **Applies when:** rolling

```
inputs {'a': 1.5, 'g': 1.6, 'theta': 1.7, 'm': 1.8, 'R': 1.9}
-> 0.375427 kg*m^2   (ran on rolling_incline_acceleration__Ir.onnx; matches the Python engine)
```

Acceleration of a body rolling without slipping down an incline — solve for rolling radius — `rolling_incline_acceleration.R`

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{mR^2}}$$

input	meaning	unit
`a`	acceleration of the center of mass along the incline	`m/s^2`
`g`	acceleration	`m/s^2`
`theta`	angle of the incline	`rad`
`Ir`	moment of inertia about the center	`kg*m^2`
`m`	mass	`kg`

- **Output unit:** `m`
- **File:** `rolling_incline_acceleration__R.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** rolling

```
inputs {'a': 1.5, 'g': 1.6, 'theta': 1.7, 'Ir': 1.8, 'm': 1.9}
-> -4.04936, 4.04936 m   (ran on rolling_incline_acceleration__R.onnx; matches the Python engine)
```

Acceleration of a body rolling without slipping down an incline — solve for acceleration of the center of mass along the incline — `rolling_incline_acceleration.a`

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{mR^2}}$$

input	meaning	unit
`g`	acceleration	`m/s^2`
`theta`	angle of the incline	`rad`
`Ir`	moment of inertia about the center	`kg*m^2`
`m`	mass	`kg`
`R`	rolling radius	`m`

- **Output unit:** `m/s^2`
- **File:** `rolling_incline_acceleration__a.onnx`
- **Applies when:** rolling

```
inputs {'g': 2.0, 'theta': 2.1, 'Ir': 2.2, 'm': 2.3, 'R': 2.4}
-> 1.48055 m/s^2   (ran on rolling_incline_acceleration__a.onnx; matches the Python engine)
```

Acceleration of a body rolling without slipping down an incline — solve for gravity — `rolling_incline_acceleration.g`

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{mR^2}}$$

input	meaning	unit
`a`	acceleration of the center of mass along the incline	`m/s^2`
`theta`	angle of the incline	`rad`
`Ir`	moment of inertia about the center	`kg*m^2`
`m`	mass	`kg`
`R`	rolling radius	`m`

- **Output unit:** `m/s^2`
- **File:** `rolling_incline_acceleration__g.onnx`
- **Applies when:** rolling

```
inputs {'a': 2.0, 'theta': 2.1, 'Ir': 2.2, 'm': 2.3, 'R': 2.4}
-> 2.70169 m/s^2   (ran on rolling_incline_acceleration__g.onnx; matches the Python engine)
```

Acceleration of a body rolling without slipping down an incline — solve for mass — `rolling_incline_acceleration.m`

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{mR^2}}$$

input	meaning	unit
`a`	acceleration of the center of mass along the incline	`m/s^2`
`g`	acceleration	`m/s^2`
`theta`	angle of the incline	`rad`
`Ir`	moment of inertia about the center	`kg*m^2`
`R`	rolling radius	`m`

- **Output unit:** `kg`
- **File:** `rolling_incline_acceleration__m.onnx`
- **Applies when:** rolling

```
inputs {'a': 1.5, 'g': 1.6, 'theta': 1.7, 'Ir': 1.8, 'R': 1.9}
-> 8.63017 kg   (ran on rolling_incline_acceleration__m.onnx; matches the Python engine)
```

Acceleration of a body rolling without slipping down an incline — solve for angle of the incline — `rolling_incline_acceleration.theta`

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{mR^2}}$$

input	meaning	unit
`a`	acceleration of the center of mass along the incline	`m/s^2`
`g`	acceleration	`m/s^2`
`Ir`	moment of inertia about the center	`kg*m^2`
`m`	mass	`kg`
`R`	rolling radius	`m`

- **Output unit:** `rad`
- **File:** `rolling_incline_acceleration__theta.onnx`
- **Branches:** 2 — take the one that is positive
- **Applies when:** rolling

Body-frame precession rate of a torque-free symmetric body — solve for moment of inertia about a transverse axis — `torque_free_precession.I1`

$$\Omega_p = \frac{\omega_3 (I_3 - I_1)}{I_1}$$

input	meaning	unit
`Wp`	precession rate seen in the body frame	`rad/s`
`w3`	spin angular velocity about the symmetry axis	`rad/s`
`I3`	moment of inertia about the symmetry axis	`kg*m^2`

- **Output unit:** `kg*m^2`
- **File:** `torque_free_precession__I1.onnx`
- **Applies when:** torque_free

```
inputs {'Wp': 2.0, 'w3': 2.1, 'I3': 2.2}
-> 1.12683 kg*m^2   (ran on torque_free_precession__I1.onnx; matches the Python engine)
```

Body-frame precession rate of a torque-free symmetric body — solve for moment of inertia about the symmetry axis — `torque_free_precession.I3`

$$\Omega_p = \frac{\omega_3 \left(I_3 - I_1\right)}{I_1}$$

input	meaning	unit
`Wp`	precession rate seen in the body frame	`rad/s`
`w3`	spin angular velocity about the symmetry axis	`rad/s`
`I1`	moment of inertia about a transverse axis	`kg*m^2`

- **Output unit:** `kg*m^2`
- **File:** `torque_free_precession__I3.onnx`
- **Applies when:** torque_free

```
inputs {'Wp': 2.0, 'w3': 2.1, 'I1': 2.2}
-> 4.29524 kg*m^2   (ran on torque_free_precession__I3.onnx; matches the Python engine)
```

Body-frame precession rate of a torque-free symmetric body — solve for precession rate seen in the body frame — `torque_free_precession.Wp`

$$\Omega_p = \frac{\omega_3 \left(I_3 - I_1\right)}{I_1}$$

input	meaning	unit
`w3`	spin angular velocity about the symmetry axis	`rad/s`
`I1`	moment of inertia about a transverse axis	`kg*m^2`
`I3`	moment of inertia about the symmetry axis	`kg*m^2`

- **Output unit:** `rad/s`
- **File:** `torque_free_precession__Wp.onnx`
- **Applies when:** torque_free

```
inputs {'w3': 2.0, 'I1': 2.1, 'I3': 2.2}
-> 0.0952381 rad/s   (ran on torque_free_precession__Wp.onnx; matches the Python engine)
```

Body-frame precession rate of a torque-free symmetric body — solve for spin angular velocity about the symmetry axis — `torque_free_precession.w3`

$$\Omega_p = \frac{\omega_3 \left(I_3 - I_1\right)}{I_1}$$

input	meaning	unit
`Wp`	precession rate seen in the body frame	`rad/s`
`I1`	moment of inertia about a transverse axis	`kg*m^2`
`I3`	moment of inertia about the symmetry axis	`kg*m^2`

- **Output unit:** `rad/s`
- **File:** `torque_free_precession__w3.onnx`
- **Applies when:** torque_free

```
inputs {'Wp': 2.0, 'I1': 2.1, 'I3': 2.2}
-> 42 rad/s   (ran on torque_free_precession__w3.onnx; matches the Python engine)
```

Orbital speed on an elliptical orbit (vis-viva) — solve for constant — `vis_viva.G`

$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

input	meaning	unit
`v`	orbital speed at distance r	`m/s`
`M`	mass of the central body	`kg`
`r`	distance from the center of the central body	`m`
`a`	semi-major axis	`m`

- **Output unit:** `N*m^2/kg^2`
- **File:** `vis_viva__G.onnx`
- **Applies when:** orbit

```
inputs {'v': 2.0, 'M': 2.1, 'r': 2.2, 'a': 2.3}
-> 4.01587 N*m^2/kg^2   (ran on vis_viva__G.onnx; matches the Python engine)
```

Orbital speed on an elliptical orbit (vis-viva) — solve for mass of the central body — `vis_viva.M`

$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

input	meaning	unit
`v`	orbital speed at distance r	`m/s`
`G`	gravitational constant	`N*m^2/kg^2`
`r`	distance from the center of the central body	`m`
`a`	semi-major axis	`m`

- **Output unit:** `kg`
- **File:** `vis_viva__M.onnx`
- **Applies when:** orbit

```
inputs {'v': 2.0, 'G': 2.1, 'r': 2.2, 'a': 2.3}
-> 4.01587 kg   (ran on vis_viva__M.onnx; matches the Python engine)
```

Orbital speed on an elliptical orbit (vis-viva) — solve for semi-major axis — `vis_viva.a`

$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

input	meaning	unit
`v`	orbital speed at distance r	`m/s`
`G`	gravitational constant	`N*m^2/kg^2`
`M`	mass of the central body	`kg`
`r`	distance from the center of the central body	`m`

- **Output unit:** `m`
- **File:** `vis_viva__a.onnx`
- **Applies when:** orbit

```
inputs {'v': 2.0, 'G': 2.1, 'M': 2.2, 'r': 2.3}
-> 265.65 m   (ran on vis_viva__a.onnx; matches the Python engine)
```

Orbital speed on an elliptical orbit (vis-viva) — solve for distance from the center of the central body — `vis_viva.r`

$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

input	meaning	unit
`v`	orbital speed at distance r	`m/s`
`G`	gravitational constant	`N*m^2/kg^2`
`M`	mass of the central body	`kg`
`a`	semi-major axis	`m`

- **Output unit:** `m`

- **File:** `vis_viva__r.onnx`
- **Applies when:** orbit

```
inputs {'v': 2.0, 'G': 2.1, 'M': 2.2, 'a': 2.3}
-> 1.53777 m   (ran on vis_viva__r.onnx; matches the Python engine)
```

Orbital speed on an elliptical orbit (vis-viva) — solve for orbital speed at distance r — `vis_viva.v`

$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

input	meaning	unit
`G`	gravitational constant	`N*m^2/kg^2`
`M`	mass of the central body	`kg`
`r`	distance from the center of the central body	`m`
`a`	semi-major axis	`m`

- **Output unit:** `m/s`
- **File:** `vis_viva__v.onnx`
- **Applies when:** orbit

```
inputs {'G': 2.0, 'M': 2.1, 'r': 2.2, 'a': 2.3}
-> 1.41142 m/s   (ran on vis_viva__v.onnx; matches the Python engine)
```